Basics of Running Jobs

NERSC uses Slurm for cluster/resource management and job scheduling. Slurm is responsible for allocating resources to users, providing a framework for starting, executing and monitoring work on allocated resources and scheduling work for future execution.

Additional Resources

- Documentation: https://slurm.schedmd.com/documentation.html
- Tutorial: https://slurm.schedmd.com/tutorials.html
- Manual: https://slurm.schedmd.com/man_index.html
- FAQ: https://slurm.schedmd.com/faq.html

Jobs

A **job** is an allocation of resources such as compute nodes assigned to a user for an amount of time. Jobs can be interactive or batch (e.g., a script) scheduled for later execution.

> **Tip**
>
> NERSC provides an extensive set of example job scripts

Once a job is assigned a set of nodes, the user is able to initiate parallel work in the form of job steps (sets of tasks) in any configuration within the allocation.

When you log in to a NERSC system you land on a login node. Login nodes are for editing, compiling, or preparing jobs. They are not for running jobs. From the login node you can interact with Slurm to submit job scripts or start interactive jobs.

NERSC's environment is configured to support a diverse workload including high-throughput serial tasks, full system capability simulations and complex workflows.

Submitting jobs

sbatch

`sbatch` is used to submit a job script for later execution. The script will typically contain one or more `srun` commands to launch parallel tasks.

When you submit the job, Slurm responds with the job's ID, which will be used to identify this job in reports from Slurm.

```
$ sbatch first-job.sh
Submitted batch job 864933
```

Slurm checks your file system usage for quota enforcment at job submission time and will reject the job if you are over your quota.

salloc

`salloc` is used to allocate resources for a job in real time as an interactive batch job. Typically this is used to allocate resources and spawn a shell. The shell is then used to execute `srun` commands to launch parallel tasks.

srun

`srun` is used to submit a job for execution or initiate job steps in real time. A job can contain multiple job steps executing sequentially or in parallel on independent or shared resources within the job's node allocation. This command is typically executed within a script which is submitted with `sbatch` or from an interactive prompt on a compute node obtained via `salloc`.

Options

At a minimum, a job script must include the number of nodes, time, type of nodes (constraint), quality of service (QOS), and for Perlmutter GPU jobs, the number of GPUs. If a script does not specify any of these options, then a default may be applied.

Tip

It is good practice to always set the account option (`--account=<NERSC Project>`).

Information about available NERSC projects is avaliable on iris and via the `iris` command on NERSC systems.

The full list of directives is documented in the man pages for the `sbatch` command (see `man sbatch`). Each option can be specified either as a directive in the job script:

```
#!/bin/bash
#SBATCH -N 2
```

Or as a command line option when submitting the script:

```
sbatch -N 2 ./first-job.sh
```

The command line and directive versions of an option are equivalent and interchangeable. If the same option is present both on the command line and as a directive, the command line will be honored. If the same option or directive is specified twice, the last value supplied will be used.

Also, many options have both a long form, e.g., `--nodes=2` and a short form, e.g., `-N 2`. These are equivalent and interchangable.

Many options are common to both `sbatch` and `srun`, for example `sbatch -N 4 ./first-job.sh` allocates 4 nodes to `first-job.sh`, and `srun -N 4 uname -n` inside the job runs a copy of `uname -n` on each of 4 nodes. If you don't specify an option in the `srun` command line, `srun` will inherit the value of that option from `sbatch`.

In these cases the default behavior of `srun` is to assume the same options as were passed to `sbatch`. This is achieved via environment variables: `sbatch` sets a number of environment variables with names like `SLURM_JOB_NUM_NODES` and srun checks the values of those variables. This has two important consequences:

1. Your job script can see the settings it was submitted with by checking these environment variables
2. You should not override these environment variables. Also be aware that if your job script does certain tricky things, such as using ssh to launch a command on another node, the environment might not be propagated and your job may not behave correctly

Commonly Used Options

The below table lists some commonly used `sbatch` / `salloc` / `srun` options as well as their meaning. All the listed options can be used with the `sbatch` or `salloc` commands (either on the command line or as directives within a script). Many are also commonly used with `srun` within a script or interactive job.

The long and short forms of each option are interchangeable, but their formats differ. The long form begins with a double hyphen (--) and includes a word, acronym, or phrase (with words separated by single hyphens (-) followed by an equals sign and any argument to the option (e.g., `--time=10:00:00`), whereas the short form consists of a single hyphen (-) and a single letter, followed by a space and any argument to the option (e.g., `-t 10:00:00`). For clarity, we

recommend using the long form for Slurm directives in a script. This makes it easier to understand what options are being set on each line.

Option (long form)	Option (short form)	Meaning	sbatch/salloc?	srun?
`--time`	`-t`	maximum walltime	Y	N
`--time-min`	(none)	minimum walltime	Y	N
`--nodes`	`-N`	number of nodes	Y	Y
`--ntasks`	`-n`	number of MPI tasks	Y	Y
`--cpus-per-task`	`-c`	number of processors per MPI task	Y	Y
`--gpus`	`-G`	total number of GPUs (Perlmutter)	Y	Y
`--gpus-per-node`	(none)	number of GPUs per node (Perlmutter)	Y	Y
`--gpus-per-task`	(none)	number of GPUs per MPI task (Perlmutter)	Y	Y
`--constraint`	`-C`	constraint (e.g., type of resource)	Y	N
`--qos`	`-q`	quality of service (QOS)	Y	N
`--account`	`-A`	project to charge for this job	Y	N
`--licenses`	`-L`	licenses (filesystem	Y	N

Option (long form)	Option (short form)	Meaning	sbatch/salloc?	srun?
		required for job)		
`--job-name`	`-J`	name of job	Y	N

Writing a Job Script

A clear job script will include at least the number of nodes, walltime, type of nodes (constraint), quality of service (QOS), and for Perlmutter GPU jobs, the number of GPUs. These options could be specified on the command line, but for clarity and to establish a record of the job submission, we recommend including all these options (and more) in your job script.

A Slurm job script begins with a shell invocation (e.g., `#!/bin/bash`) followed by lines of directives, each of which begins with `#SBATCH`. After these directives, users then include the commands to be run in the script, including the setting of environment variables and the setup of the job. Usually (but not always) the script includes at least one `srun` command, launching a parallel job onto one or more nodes allocated to the job.

```
#!/bin/bash
#SBATCH --nodes=<nnodes>
#SBATCH --time=hh:mm:ss
#SBATCH --constraint=<architecture>
#SBATCH --qos=<QOS>
#SBATCH --account=<project_name>

# set up for problem & define any environment variables here

srun -n <num_mpi_processes> -c <cpus_per_task> a.out

# perform any cleanup or short post-processing here
```

The above script is easily applied only to the simplest of cases and is not widely generalizable. In this simple case, a user would replace the items between `< >` with specific arguments, e.g., `--nodes=2` or `--qos=debug`. The format for the maximum walltime request is number of hours, number of minutes, and number of seconds, separated by colons (e.g., `--time=12:34:56` for 12 hours, 34 minutes, and 56 seconds).

There are many factors to consider when creating a script for your particular job. In our experience, we find that determining the correct settings for number of CPUs per task, process affinity, etc. can be tricky. Consequently, we recommend using the Job Script Generator to generate the correct `#SBATCH` directives, `srun` arguments, and process affinity settings for you.

The job script generator will provide the correct runtime arguments for your job, but may not adequately demonstrate a way to run jobs that fits your particular workflow or application. To help with this, we have developed a curated collection of example job scripts for users to peruse for inspiration.

Defaults

If you do not specify the following options in your script, defaults will be assigned.

Option	Default
nodes	1
time	10 minutes
qos	debug
account	set in Iris

You can also set the default account that is charged for your jobs by setting environment variables. You can use `SBATCH_ACCOUNT=<account_name>` and `SALLOC_ACCOUNT=<account_name>` to set a default account to charge for your sbatch and salloc jobs, respectively. These can be set like so:

```
export SBATCH_ACCOUNT=mxxxx
export SALLOC_ACCOUNT=mxxxx
```

There is no default architecture

Jobs not specifying the "constraint" will be rejected.

When using srun on GPU nodes, you must explicitly request for GPU resources

One must use the `--gpus` or `-G` flag to make the allocated node's GPUs visible to your `srun` command. You may also use the `--gpus-per-task` flag to set the number of GPUs per MPI task.

Failing to do so, one may get errors / complaints similar to:

```
no CUDA-capable device is detected
```

```
No Cuda device found
```

Debugging issues

If there are issues with job submission check:

- that all required options are set
- that selected options match our queue policies
- that appropriate modules (see Modules) are loaded
- your account balance (iris)
- your compliance with quota requirements
- NERSC Center Status for any current issues

Available memory for applications on compute nodes

Some memory on compute nodes is reserved for the operating system. The amount of memory reported below for each type of node represents the amount of physical memory installed, but the memory that users can use will be around 5-10 GB less, due to operating system processes, file system caches, etc.

Node Type	Total Memory (GB)
Perlmutter GPU	CPU: 256, GPU: 160
Perlmutter CPU	CPU: 512

Quota Enforcement

User job submissions will be rejected by Slurm if the user has exceeded their space or inode quota in their scratch or home directories. This quota check is performed twice: first when the job is submitted and again when the running job invokes `srun`. This could mean that if you went over quota after submitting the job, the job could fail when it runs. Please check your quota regularly and delete or archive data as needed.

Queue Wait Times

Queue wait times for past jobs can be a useful guide in estimating wait times of current jobs. The wait time depends on the quality of service (QOS), requested resources (nodes, time, filesystems, etc), jobs in the queue, your other jobs and other jobs from the same NERSC project.

For active jobs in the queue, you can monitor their start times with the `squeue --start` command. In the below example, job `1448935` can't start because the user has exceeded the maximum jobs per QOS limit. Slurm will report `N/A` for the start time estimate if nodes are not currently being reserved by the scheduler for the job to run on. You can periodically check on the job to see if there is a job start time estimate.

```
$ squeue --start -j 1448935
             JOBID PARTITION     NAME     USER ST          START_TIME  NODES
SCHEDNODES           NODELIST(REASON)
           1448935    shared sharejob    elvis PD                 N/A      1
(null)               (QOSMaxJobsPerUserLimit)
```

Tip

`sqs` is an alias to `squeue` with predefined helpful options, including `--start`.

In most cases, jobs are in the pending state due to their low priority level; Slurm indicates this with `(Priority)` in the output, as shown below.

```
$ squeue --start -j 56789012
             JOBID PARTITION     NAME     USER ST          START_TIME  NODES
SCHEDNODES           NODELIST(REASON)
          56789012    shared sharejob    elvis PD                 N/A      1
(null)               (Priority)
```

For more details on job state and reason codes, please see:

- https://slurm.schedmd.com/squeue.html#lbAF
- https://slurm.schedmd.com/squeue.html#lbAG

Heterogeneous Jobs

It is possible for users to run a single job that includes multiple types of nodes (e.g., CPU-only and GPU nodes in a single job). More information is available in the examples for heterogeneous jobs.

Further reading about jobs

- Interactive jobs
- I/O Performance
- Example jobs
- Monitoring jobs
- Best Practices for jobs
- Troubleshooting Slurm